灵巧手手内物体旋转项目
环境复现与跨机器迁移报告

In-Hand Cube Rotation with LEAP Hand — MjLab

项目仓库：github.com/Msornerrrr/in-hand-rotation-mjlab

本报告用途

记录已验证成功的 RTX 5060 Laptop + Ubuntu 22.04 复现环境，沉淀依赖版本、排错过程和迁移到其他计算机时的标准安装流程。

状态：环境部署与项目复现步骤已跑通

日期：2026 年 9 月

1. 项目与复现目标

本次复现选择的是 In-Hand Cube Rotation with LEAP Hand — MjLab 项目。项目以手内物体旋转（in-hand rotation / reorientation）为核心任务，采用 LEAP Hand、MuJoCo、MuJoCo-Warp、MjLab 与强化学习策略进行仿真训练和推理。

该项目与后续 21-DOF 灵巧手“手内旋转物体、盘核桃、reorientation、sim-to-real”研究方向高度相关，因此本次复现不仅用于验证代码，也作为后续自定义灵巧手环境开发的基础模板。

核心结论

当前 RTX 5060 Laptop（8GB VRAM）可以沿 MuJoCo + MjLab 路线运行该项目。关键不是把机器升级到某个特定 Ubuntu 版本，而是固定与项目源码匹配的 MjLab API 版本，并锁定完整 Python 依赖。

2. 已验证成功的最终环境

类别	最终版本/配置	备注
操作系统	Ubuntu 22.04 LTS	系统 Python 3.10 不修改
GPU	NVIDIA GeForce RTX 5060 Laptop GPU	8GB VRAM
GPU 架构	Blackwell	Compute Capability 12.0
CUDA 架构	sm_120	通过 torch.cuda.get_device_capability() 验证
系统 RAM	16GB	小规模训练可用，大规模环境数量受限
NVIDIA Driver	595.91.07	nvidia-smi 正常识别 GPU
Python	3.13.15	由 uv 管理项目环境
uv	0.12.16	项目/虚拟环境管理
PyTorch	2.9.0+cu128	CUDA Runtime 12.8
MjLab	1.1.1	与项目使用的旧 API 匹配
MuJoCo	3.5.0	从 PyPI 获取
MuJoCo-Warp	3.5.0.2	与该复现环境匹配
Warp	1.12.0	与 MjLab 1.1.1 兼容
SciPy	1.16.2	项目运行时需要
Triton	3.5.0	PyTorch 2.9 依赖

3. 软件栈关系：PyTorch、CUDA、GPU、uv

本次复现中经常被混淆的几个概念可以分成不同层级：

组件	作用	本项目中的角色
GPU	实际执行并行计算的硬件	RTX 5060 Laptop，Blackwell / sm_120
VRAM	GPU 的高速显存	8GB，用于保存 GPU 计算中的模型、状态和中间数据

NVIDIA Driver	操作系统与 GPU 的驱动层	595.91.07，负责让 Linux 正确使用 RTX 5060
CUDA	NVIDIA GPU 计算平台/运行时生态	PyTorch、MuJoCo-Warp 等 GPU 计算的底层基础
PyTorch	深度学习框架	策略网络、张量、训练、GPU 计算
MuJoCo	物理仿真器	LEAP Hand 与 Cube 的动力学/接触仿真
MuJoCo-Warp	MuJoCo 的 GPU 加速能力	让仿真利用 NVIDIA GPU 并行计算
MjLab	机器人强化学习框架	环境、任务、MDP、训练接口等
uv	Python 项目/依赖/虚拟环境管理器	固定 Python、安装依赖、生成 uv.lock

关键理解

“nvidia-smi 显示的 CUDA Version”不等于“项目实际使用的 PyTorch CUDA Runtime”。本次 nvidia-smi 显示 Driver 支持 CUDA 13.2，而项目内 PyTorch 实际使用 CUDA 12.8（cu128），两者可以同时成立。

4. 为什么使用 Ubuntu 22.04，而不是 20.04/24.04

Ubuntu	适用特点	本项目建议
20.04	老 Isaac Gym、ROS1 老项目较多	不作为本项目主系统
22.04	新旧机器人项目兼容性较平衡，ROS2 Humble 生态成熟	推荐
24.04	面向更新的 ROS2 Jazzy/新系统生态	可以用于新项目，但本次无需为了它迁移

本项目最终使用的是 MuJoCo + MjLab，并不要求为了 ROS 而选择 Ubuntu 22.04。22.04 的优势更多体现在实验室未来还需要 ROS2、真机部署以及其他机器人项目时，系统生态更加稳妥。

5. 正确复现步骤（当前 5060 Laptop）

5.1 安装 Ubuntu 22.04

1. 安装 Ubuntu 22.04 LTS Desktop 64-bit。

2. 安装完成后执行系统更新。

```
sudo apt update
sudo apt upgrade -y
sudo reboot
```

5.2 安装 NVIDIA Driver

通过 Ubuntu 的 ubuntu-drivers/Additional Drivers 安装，不使用 NVIDIA .run 安装包，也不选择 server 驱动。最终验证使用：

```
sudo ubuntu-drivers list
sudo apt install nvidia-driver-595-open
sudo reboot
nvidia-smi
```

本次实际结果：Driver 595.91.07，RTX 5060 Laptop 8GB。

5.3 安装基础工具

```
sudo apt install -y git curl wget build-essential cmake pkg-config unzip
```

5.4 安装 uv 与 Python 3.13

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv python install 3.13
```

Ubuntu 22.04 自带 Python 3.10.12 不修改。进入项目后使用 uv 管理独立 Python 3.13 环境。

```
cd ~/Projects/in-hand-rotation-mjlab
uv python pin 3.13
uv run python --version
```

5.5 获取项目

```
cd ~/Projects
git clone https://github.com/Msornerrrr/in-hand-rotation-mjlab.git
cd in-hand-rotation-mjlab
```

6. 最终验证过的 pyproject.toml 核心版本

由于原仓库没有将 MjLab API 版本固定到与项目源码完全匹配的版本，直接 uv sync 会拉取过新的 MjLab。因此本次复现将关键依赖固定如下。

```
[project]
name = "in-hand-rotation-mjlab"
version = "0.1.0"
description = "In-hand cube rotation task using LEAP hand in MuJoCo with mjlab"
requires-python = ">=3.13,<3.14"
dependencies = [
    "mjlab[cu128]==1.1.1",
    "pyzmq>=25",
    "mujoco==3.5.0",
    "mujoco-warp==3.5.0.2",
    "warp-lang==1.12.0",
    "torch==2.9.0+cu128",
    "scipy==1.16.2",
]

[tool.uv]
package = true

[[tool.uv.index]]
name = "pytorch-cu128"
url = "https://download.pytorch.org/whl/cu128"
explicit = true

[tool.uv.sources]
torch = { index = "pytorch-cu128" }
```

实际项目文件还保留原仓库的 build-system 和 setuptools package 配置。

7. 生成 lock 文件并安装

```
rm -f uv.lock
uv lock
uv sync
```

首次安装如果 Triton 或其他较大 wheel 下载超时：

```
uv cache clean triton
UV_NATIVE_TLS=true \
UV_HTTP_TIMEOUT=600 \
UV_HTTP_RETRIES=10 \
uv sync
```

安装完成后不要立即运行 play.py，应先做版本与 GPU 检查。

8. 环境验证清单

8.1 检查版本

```
uv run python -c "import importlib.metadata as m; print('mjlab:', m.version('mjlab'));
print('mujoco:', m.version('mujoco')); print('mujoco-warp:', m.version('mujoco-warp'));
print('warp-lang:', m.version('warp-lang')); print('torch:', m.version('torch'));
print('scipy:', m.version('scipy')); print('triton:', m.version('triton'))"
```

检查项	期望值
Python	3.13.x
MjLab	1.1.1
MuJoCo	3.5.0
MuJoCo-Warp	3.5.0.2
Warp	1.12.0
PyTorch	2.9.0+cu128
SciPy	1.16.2
Triton	3.5.0

8.2 检查项目 API

```
uv run python -c "from mjlab.actuator import DelayedActuatorCfg; print('DelayedActuatorCfg:
OK')"
uv run python -c "from mjlab.envs import mdp; print('randomize_field:', hasattr(mdp,
'randomize_field'))"
```

两个检查都应成功。它们是区分 MjLab 1.1.x / 1.2.x / 1.6.x 兼容性的快速“版本指纹”。

8.3 检查 GPU/CUDA

```
uv run python -c "import torch; print('PyTorch:', torch.__version__); print('CUDA:',
torch.version.cuda); print('CUDA available:', torch.cuda.is_available()); print('GPU:',
torch.cuda.get_device_name(0)); print('Capability:', torch.cuda.get_device_capability(0))"
```

成功标志：CUDA available 为 True，GPU 为 RTX 5060 Laptop GPU，Capability 为 (12, 0)，即 sm_120。

9. 项目运行与复现顺序

9.1 先做 checkpoint 检查

```
ls -lh ckpts/
```

应至少存在：leap_left_custom_model_4900.pt。

9.2 随机策略：只验证环境是否能启动

```
uv run python scripts/play.py \
  Mjlab-Leap-Left-HandCube-Rotate \
  --agent random
```

随机策略不以成功旋转 Cube 为目标。关注的是 MuJoCo 窗口、LEAP Hand、Cube、仿真循环和 CUDA/Warp 是否正常。

9.3 预训练策略：真正验证任务

```
uv run python scripts/play.py \
  Mjlab-Leap-Left-Custom-HandCube-Rotate \
  --checkpoint-file ckpts/leap_left_custom_model_4900.pt
```

建议另开终端观察 GPU：

```
watch -n 1 nvidia-smi
```

10. RTX 5060 上的训练建议

作者示例可使用较大的并行环境数量，但 RTX 5060 Laptop 仅有 8GB VRAM、16GB RAM，不应直接套用 4096 environments。应逐级测试：

阶段	并行环境	目的
1	128	确认训练循环能稳定运行
2	256	观察显存和 RAM
3	512	提高 GPU 并行度
4	1024	寻找笔记本稳定上限

```
uv run python scripts/train.py \
  Mjlab-Leap-Left-HandCube-Rotate \
  --env.scene.num-envs 128
```

同时观察：watch -n 1 nvidia-smi，以及 watch -n 1 free -h。根据显存、RAM、GPU 利用率和 training FPS 决定是否提高环境数量。

11. 本次遇到的问题与解决方式

问题	根因	解决办法
老 Isaac Gym / RTX 5060 Blackwell/sm_120 不合适	老 Isaac Gym 的预编译 GPU PhysX 二进制并未面向 sm_120 编译	改用 MuJoCo + MuJoCo-Warp + MjLab 路线
系统 python3 是 3.10	Ubuntu 22.04 系统 Python	不要修改系统 Python；项目用 uv 管理 Python 3.13
MjLab 1.6.0 缺少 DelayedActuatorCfg	项目源码调用旧 API	固定 MjLab 1.1.1
MjLab 1.2.0 缺少 randomize_field	v1.2.0 API breaking change	继续固定到 MjLab 1.1.1
MuJoCo dev 版本不匹配 / mjENBL_MULTICCD	MjLab 版本与 MuJoCo-Warp API 不对应	固定 MuJoCo 3.5.0 与 MuJoCo-Warp 3.5.0.2
uv 说没有 mujoco 3.5.0	实际查询到了 py.mujoco.org 而不是普通 PyPI	使用 PyPI 正式发布版并固定版本
缺少 scipy	项目运行时使用 scipy.interpolate，依赖未充分声明	显式加入 scipy==1.16.2
Triton wheel 下载超时	网络/下载超时或缓存不完整	清理 triton 缓存并提高 UV_HTTP_TIMEOUT /

		UV_HTTP_RETRIES，必要时启用 native TLS

12. 迁移到其他电脑的标准 SOP

核心原则

迁移时带“代码 + pyproject.toml + uv.lock + .python-version + assets/checkpoints”，不要复制 .venv。系统级只重新安装适合新 GPU 的 NVIDIA Driver。

12.1 迁移时需要保存的文件

- pyproject.toml：声明项目依赖与 PyTorch CUDA 源。
- uv.lock：冻结实际解析后的依赖版本，是最重要的可复现文件。
- .python-version：固定项目使用 Python 3.13。
- src/、scripts/、assets/、ckpts/：源代码、仿真资产与模型权重。
- 不要复制 .venv/。新机器应重新由 uv 创建。

12.2 新电脑安装流程

1. 安装 Ubuntu 22.04 LTS。
2. 使用 ubuntu-drivers 安装适合新 GPU 的现代 NVIDIA Driver。
3. 用 nvidia-smi 确认 GPU 可见。
4. 安装 git/curl/wget/build-essential/cmake/pkg-config/unzip。
5. 安装 uv。
6. uv python install 3.13，并确认项目存在 .python-version。
7. 把项目代码和 pyproject.toml、uv.lock、.python-version 复制/clone 到新电脑。
8. 在项目目录执行 uv sync --locked。
9. 运行版本、API、CUDA 和 GPU 检查。
10. 先 random play，再 pretrained checkpoint，最后根据新 GPU 的显存和 RAM 测试训练规模。

```
cd ~/Projects/in-hand-rotation-mjlab
uv sync --locked
```

13. 迁移到 RTX 4090 24GB + 64GB RAM 的建议

4090 训练机不需要为了本项目重新设计 Python 依赖。系统层面的 NVIDIA Driver 可以根据 4090 和 Ubuntu 22.04 使用当前稳定驱动；Python 依赖则优先用已有 uv.lock 恢复。

项目角色	RTX 5060 Laptop 8GB + 16GB RAM	RTX 4090 24GB + 64GB RAM
主要用途	开发、调试、环境验证、小规模训练	正式训练、大规模并行环境、数据生成
建议环境数量	128→256→512→1024 逐级测试	从 1024 往 2048/4096 等级逐级测试
显存余量	较小	明显更宽裕
系统 RAM	16GB，容易成为限制因素	64GB，更适合大规模实验
环境文件	同一套 pyproject.toml/uv.lock	同一套 pyproject.toml/uv.lock

迁移原则

如果实验室后续要在 4090 上训练，优先复制当前项目的锁定环境，而不是重新执行无锁定的最新依赖解析。使用 uv sync --locked 可以显著降低版本漂移风险。

14. 与后续 21-DOF 灵巧手研究的衔接

当前项目可以作为“第一阶段基线”：先理解 LEAP Hand 的 in-hand cube rotation，再逐步替换任务和机器人模型。建议路线：

1. 复现 LEAP Hand + Cube + RL 基线。
2. 理解 observation、action、reward、domain randomization、PPO/RSL-RL 等模块。
3. 把 Cube rotation 扩展到其他物体形状，再考虑更接近核桃的物体。
4. 替换为实验室自己的 21-DOF 灵巧手模型。
5. 根据自有手的关节范围、驱动器和接触模型重新设计 action/actuator 配置。
6. 后续加入触觉 observation 与 sim-to-real。

15. 最终迁移/复现检查清单

检查项	命令/文件	通过标准
GPU 驱动	nvidia-smi	能看到目标 NVIDIA GPU
Python	uv run python --version	3.13.x
锁文件	uv.lock	存在且与实验版本一起保存
MjLab	importlib.metadata	1.1.1
MuJoCo	importlib.metadata	3.5.0
MuJoCo-Warp	importlib.metadata	3.5.0.2
Warp	importlib.metadata	1.12.0
PyTorch	importlib.metadata	2.9.0+cu128
SciPy	importlib.metadata	1.16.2
Triton	importlib.metadata	3.5.0
DelayedActuatorCfg	from mjlab.actuator import ...	OK
randomize_field	hasattr(mjlab.envs.mdp, ...)	True
CUDA	torch.cuda.is_available()	True
GPU capability	torch.cuda.get_device_capability()	目标 GPU 对应能力值
随机环境	scripts/play.py --agent random	仿真正常启动
预训练策略	scripts/play.py --checkpoint-file ...	策略正常执行

16. 参考项目与资料

- 项目仓库：https://github.com/Msornerrrr/in-hand-rotation-mjlab
- MjLab：https://github.com/mujocolab/mjlab
- MjLab v1.1.1：https://github.com/mujocolab/mjlab/tree/v1.1.1

- uv 官方文档：https://docs.astral.sh/uv/
- PyTorch：https://pytorch.org/
- NVIDIA CUDA 文档：https://docs.nvidia.com/cuda/

17. 模型训练

uv run python scripts/train.py Mjlab-Leap-Left-HandCube-Rotate --env.scene.num-envs 128

问题 1：rsl_rl 调用 WandB API 时发生了版本兼容问题
解决：vim .venv/lib/python3.13/site-packages/rsl_rl/utils/wandb_utils.py
原来 settings=wandb.Settings(start_method="thread"), 修改 为 settings=wandb.Settings(),